

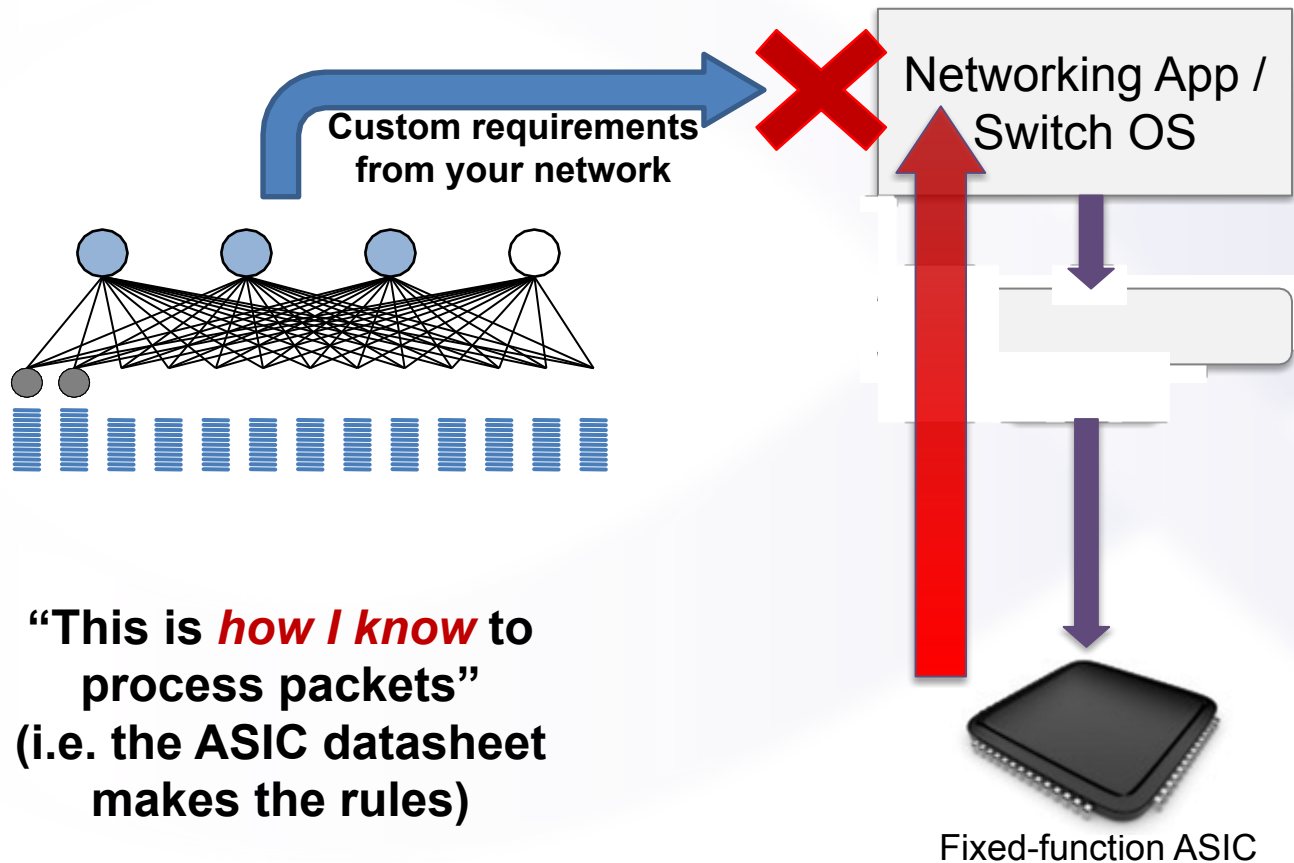
Programming The Network

SIGCOMM'16
Tutorial

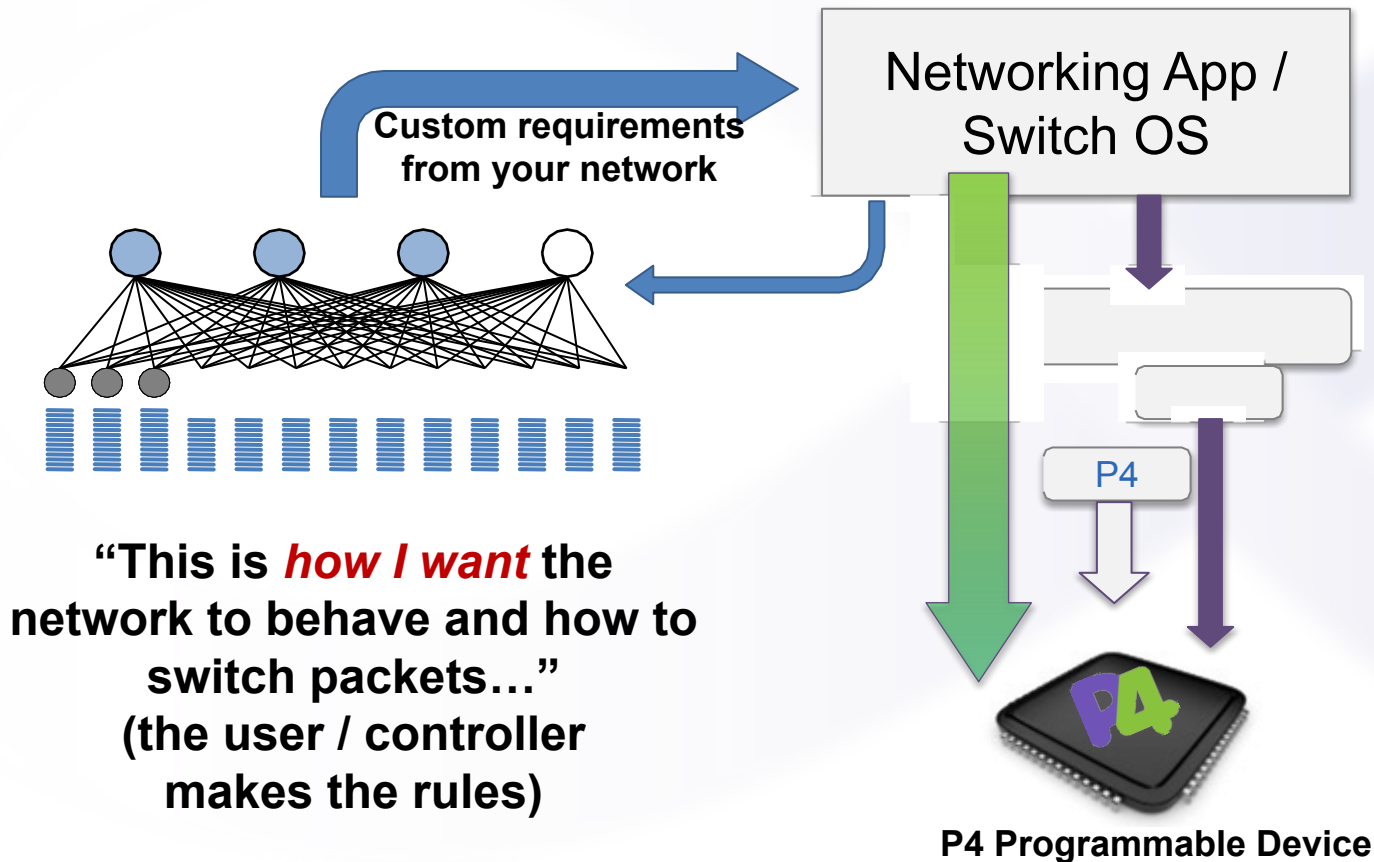
Jeongkeun “JK” Lee, Chang Kim
Aug 2016

P4 Introduction

Status Quo: Bottom-up design



A Better Approach: Top-down design



Programmable Network Devices

- **PISA: Flexible Match+Action ASICs**
 - Intel Flexpipe, Cisco Doppler, Cavium (Xpliant), Barefoot Tofino, ...
- **NPU**
 - EZchip, Netronome, ...
-
- **FPGA**
 - Xilinx, Altera, ...

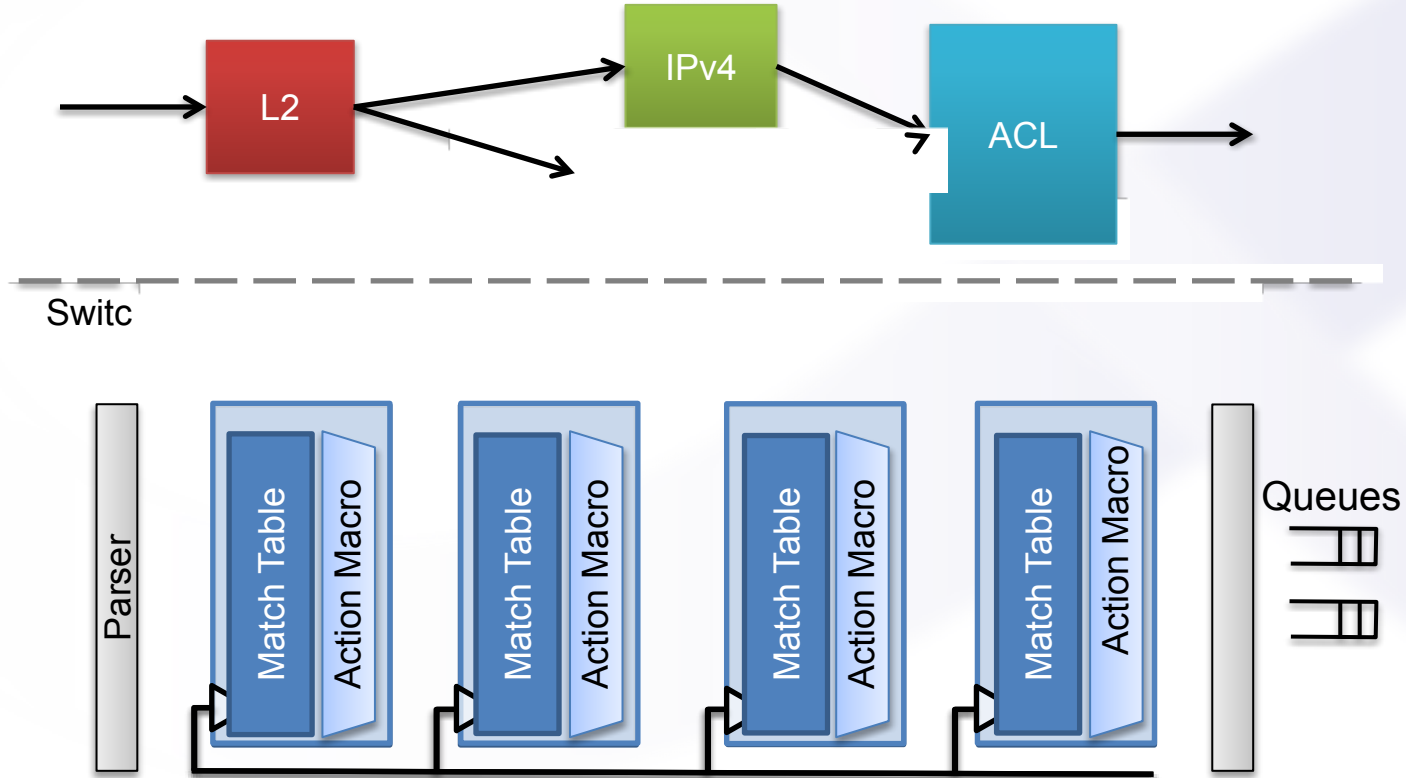
These devices let us tell them how to process packets.

Why we call it Protocol Independent Packet Processing

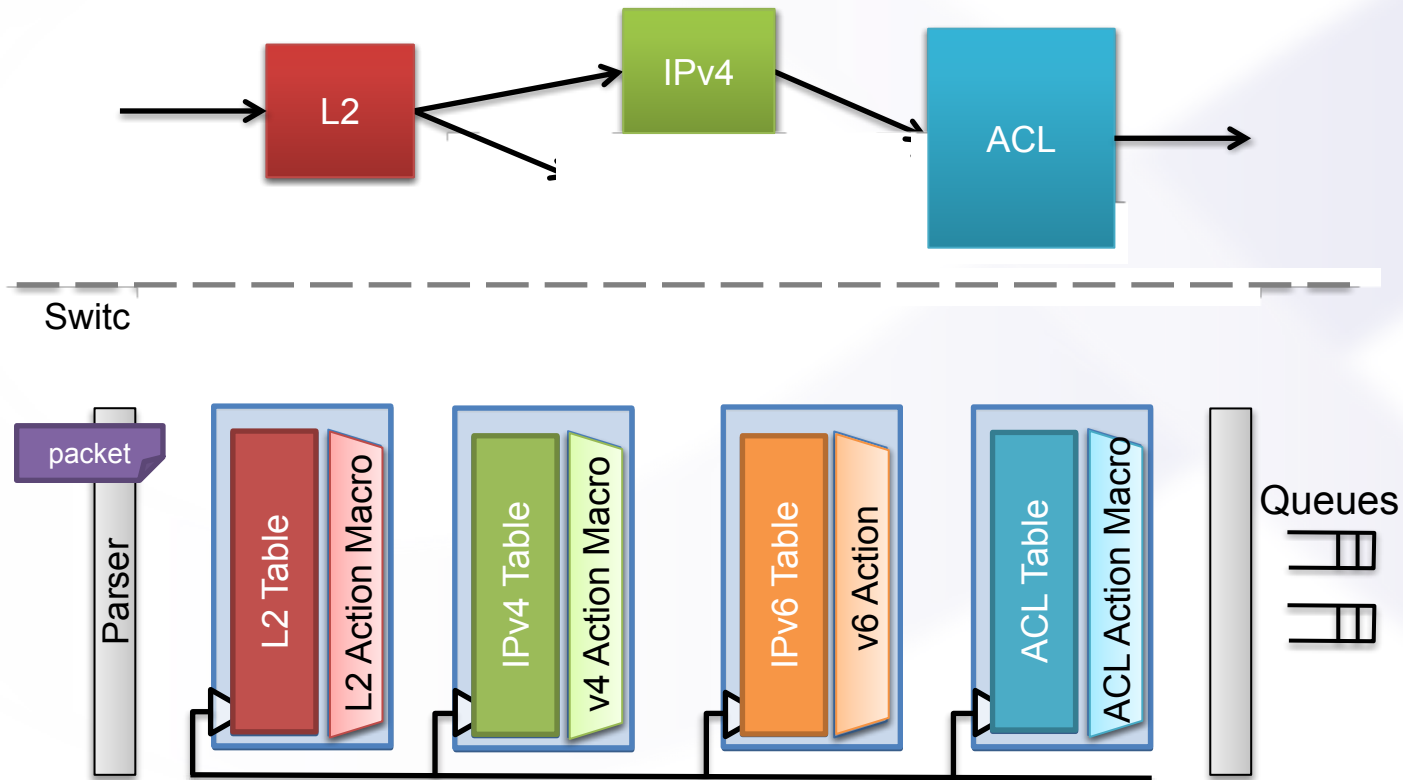
Protocol-Independent Switch Architecture (PISA)



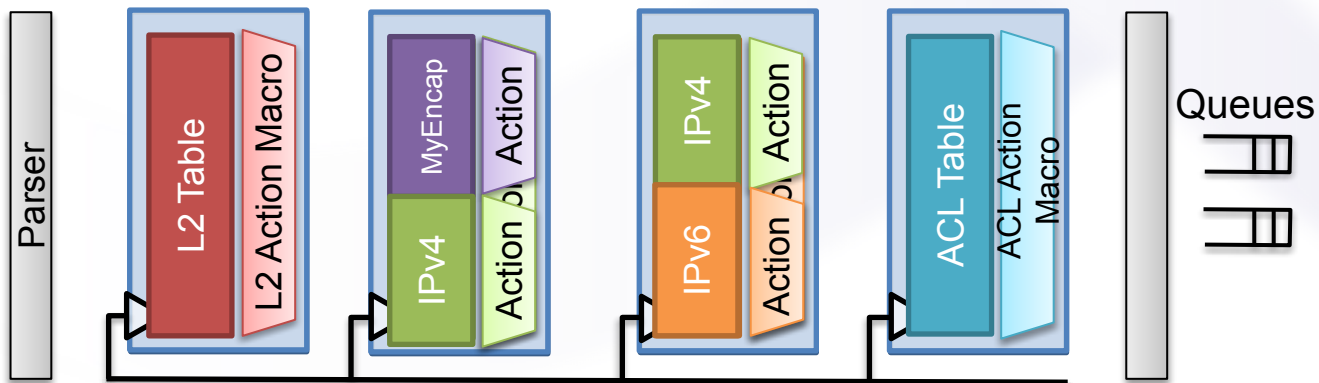
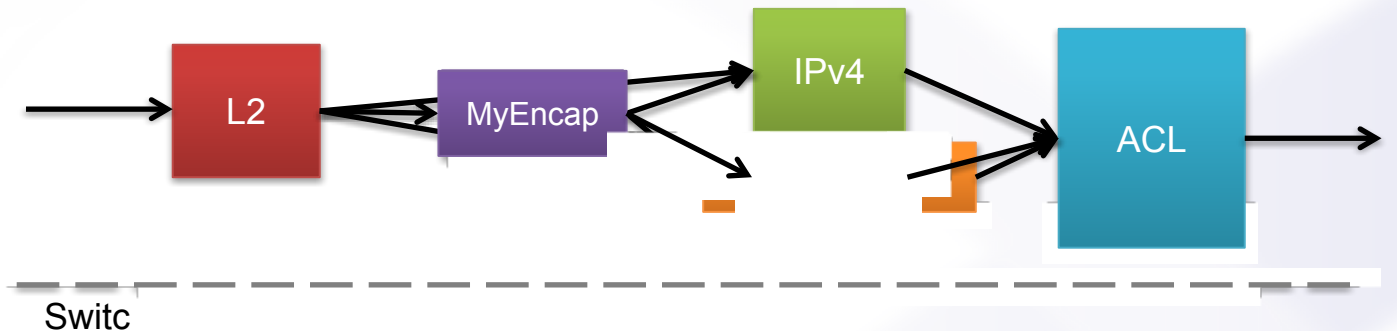
Protocol-Independent Switch Architecture (PISA)



Mapping to Physical Resources



Re-configurability



P4: Three Goals

Protocol independence

- Define a packet parser
- Define a set of typed match+action tables

Target independence

-
-

In-field Re-c

- Allow the users to change parsing and processing program in the field

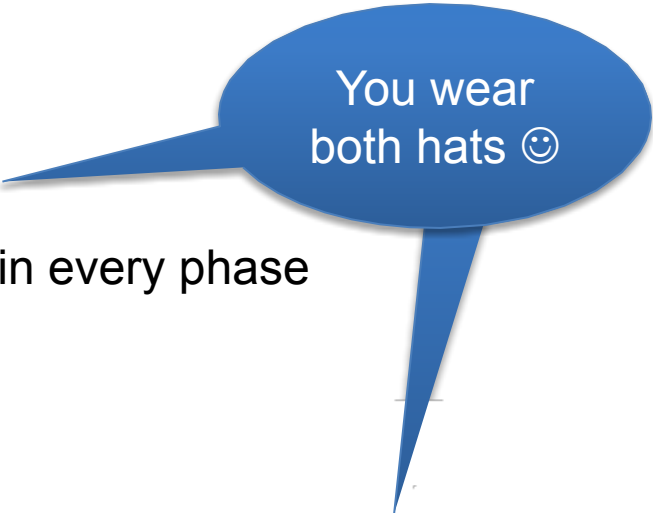
What does this mean?

- **To network device vendors**

- S/W programming practices and tools used in every phase
- Extremely fast iteration a
-
-

- **To large on-line service providers and carriers**

- No more “black boxes” in the “white boxes”
- Your devs can program, test, and debug your network devices all the way down
- You keep your own ideas



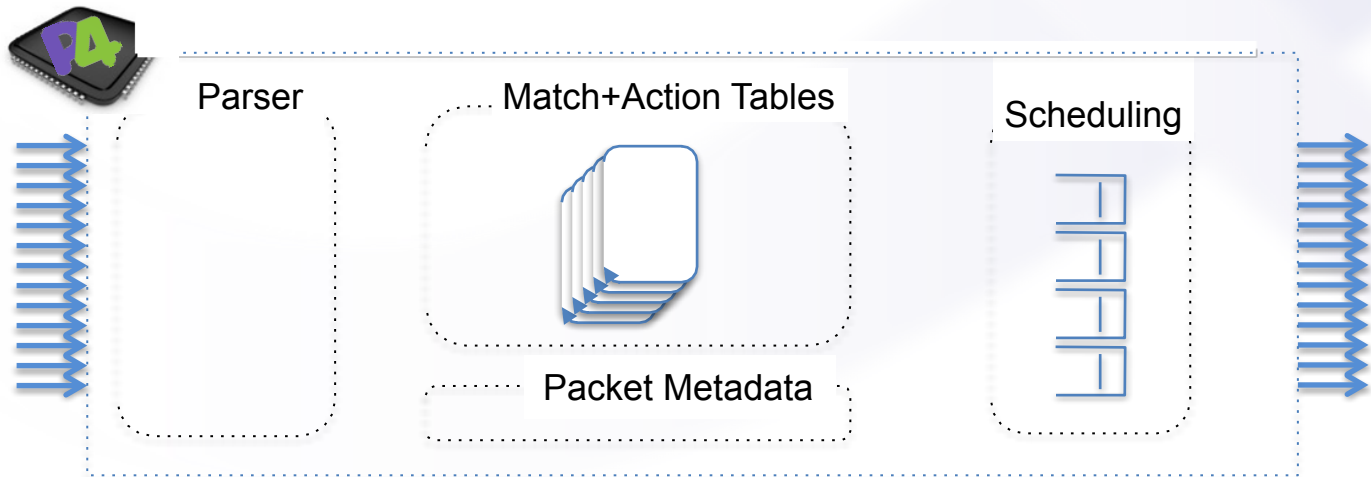
You wear
both hats 😊

Key benefits of programmable forwarding

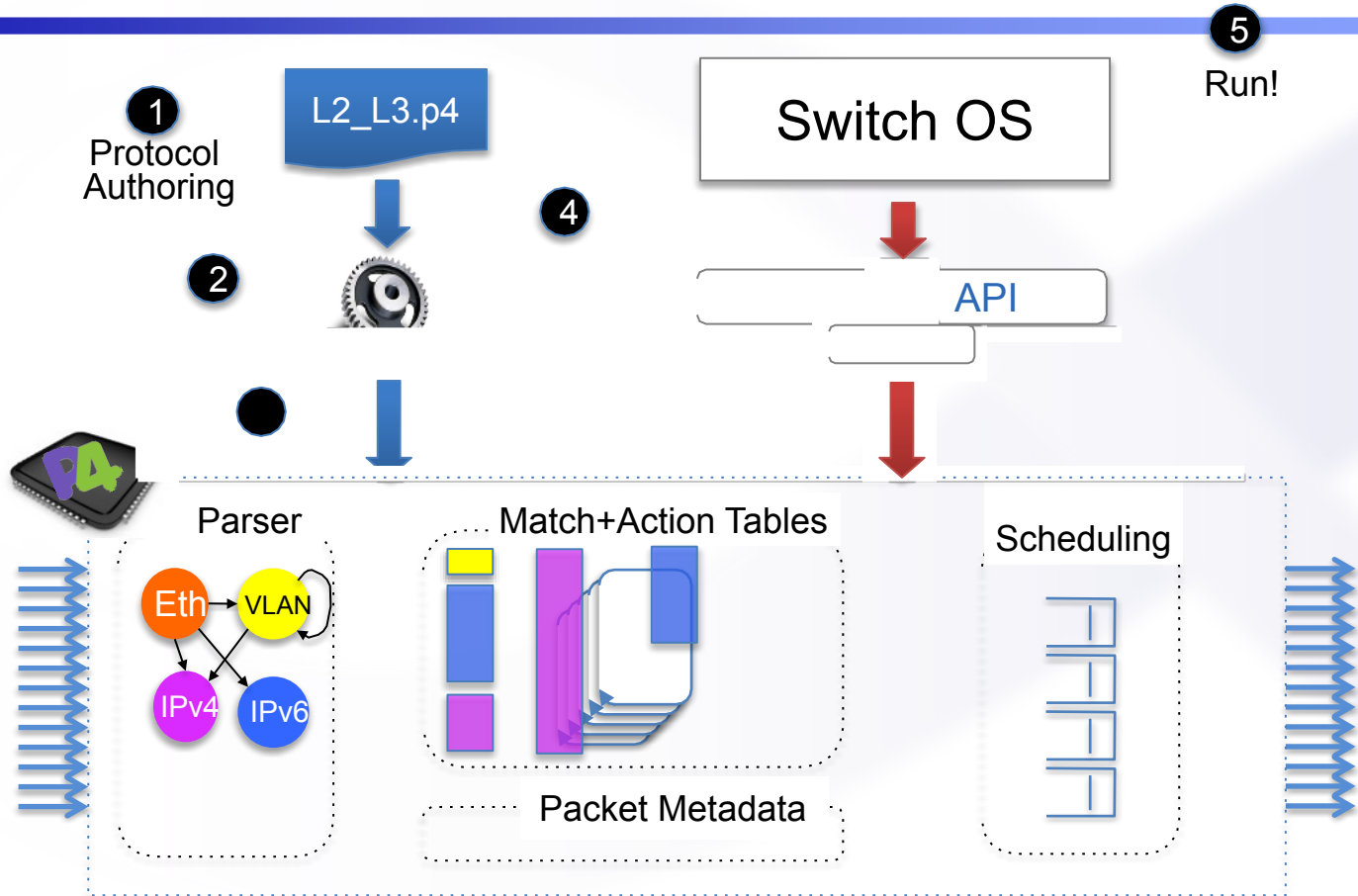
1. **New features**: Realize new protocols and behaviors very quickly
2. **Reduce complexity**: Remove unnecessary features and tables
3. **Efficient use of H/W re** _____ gest bang for buck
4. _____
5. _____
6. **Portabilit** _____ any devices
7. **Own your own network**: No need to wait for next chips or systems

P4-Based Workflow

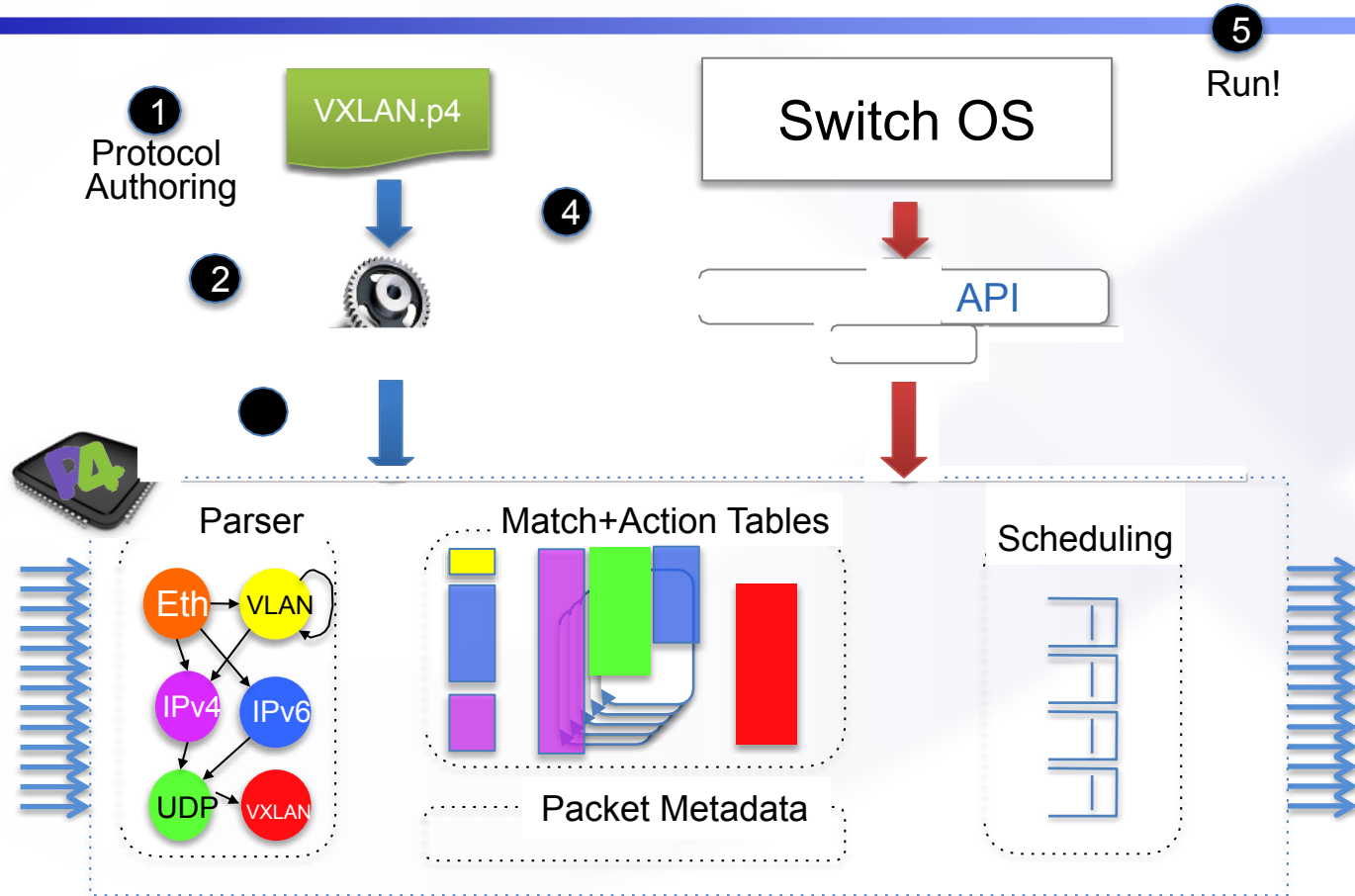
- **Device is not yet programmed**
 - Does not know about any packet formats or protocols



P4-Based Workflow



P4-Based Workflow





The P4 Language Consortium

- Consortium of academic and industry members
- Open source, evolving, and
- P4 code on GitHub
- Membership is free: contributions are welcome
- Independent, set up as a California nonprofit

The screenshot shows the P4 website with a dark header containing the logo and navigation links: SPEC, CODE, NEWS, JOIN US, and BLOG. The main content area has a large heading "It's time to say 'Hello Network'" followed by a subheading "A language to describe the data-plane of your network." Below this, there are two columns of text. The left column has two sections: "Target Independent" and "Field Reconfigurable", each with a brief description of P4's capabilities. The right column features a code snippet in a dark-themed editor showing P4 code for reading and dropping packets. At the bottom right, there is a green button with a download icon and the text "TRY IT" and "Get the code from P4factory".

Target Independent
P4 is suitable for describing everything from high-performance forwarding ASICs to software switches.

Field Reconfigurable
P4 allows network engineers to change the way their switches process packets after they are deployed.

```
reads {  
    ipv4.dstAddr : lpm;  
}  
actions {  
    do_drop;  
    route_ipv4;  
}  
size: 2048;  
}  
  
control ingress {  
    apply(routing);  
}
```

TRY IT Get the code from P4factory



P4.org Membership



Original P4 Paper Authors:



Operators



System



Targets



Academia



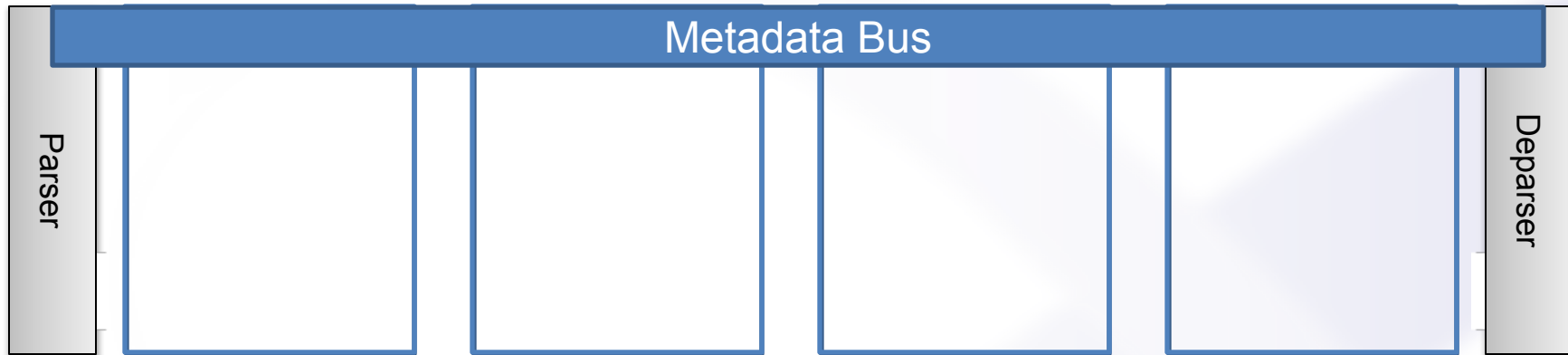
- **Open source**, evolving, domain-specific language
- Permissive Apache license, code on GitHub today

- **Membership is free**: contributions are welcome
- Independent, set up as a California nonprofit

P4 Concepts

- Pipelin

The anatomy of a basic pipeline

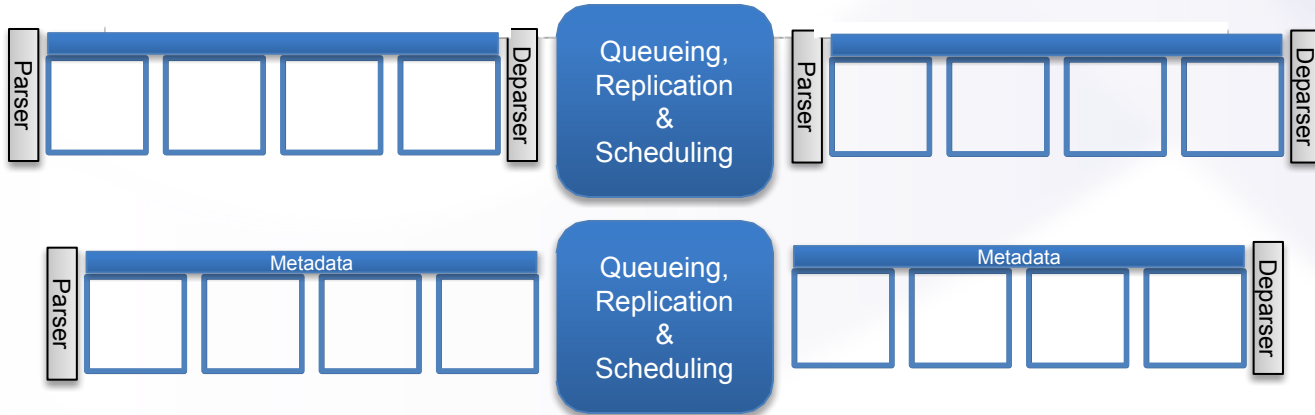


- **Parser**
 - Converts packet data into a metadata (Parsed Representation)
- **Match+Action Tables**
 - Operate on metadata
- **Deparser**
 - Converts metadata back into a serialized packet
- **Metadata Bus**
 - Carries the information within the pipeline

All are optional

Anatomy of a Switch

- Ingress Pipeline
- Egress Pipeline
- Traffic Manager
 - N:1 Relationships: Queuei
 -
 -



P4 Program Sections

program.p4

Data Declarations

```
header_type ethernet_t { ... }
header_type l2_metadata_t { ... }

header ethernet_t ethernet;
header vlan_tag_t vlan_tag[2];
metadata l2_metadata_t l2_meta;
```

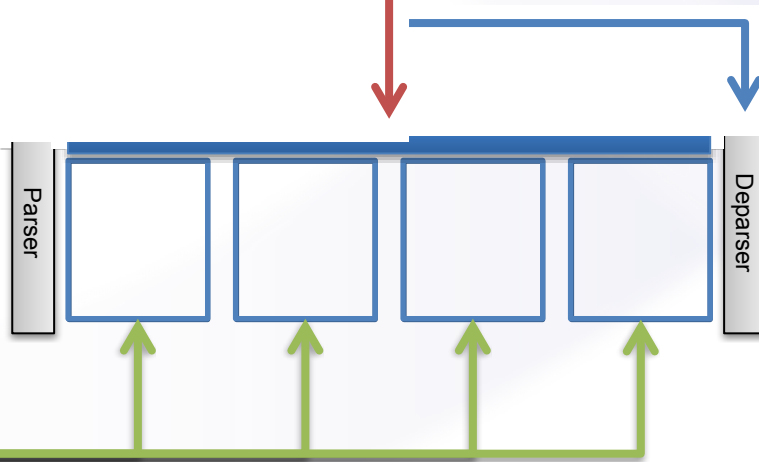
Pars

```
parser parse_ethernet {
  extract(ethernet);
  return switch(ethernet) {
    0x8100 : parse_vlan_tag;
    0x0800 : parse_ipv4;
    0x8847 : parse_mpls;
    default: ingress;
  }
}
```

Table + Control Flow Program

```
table port_table { ... }

control ingress {
  apply(port_table);
  if (l2_meta.vlan_tags == 0) {
    process_assign_vlan();
  }
}
```



P4 program defines what each table CAN do

Control Plane Roles

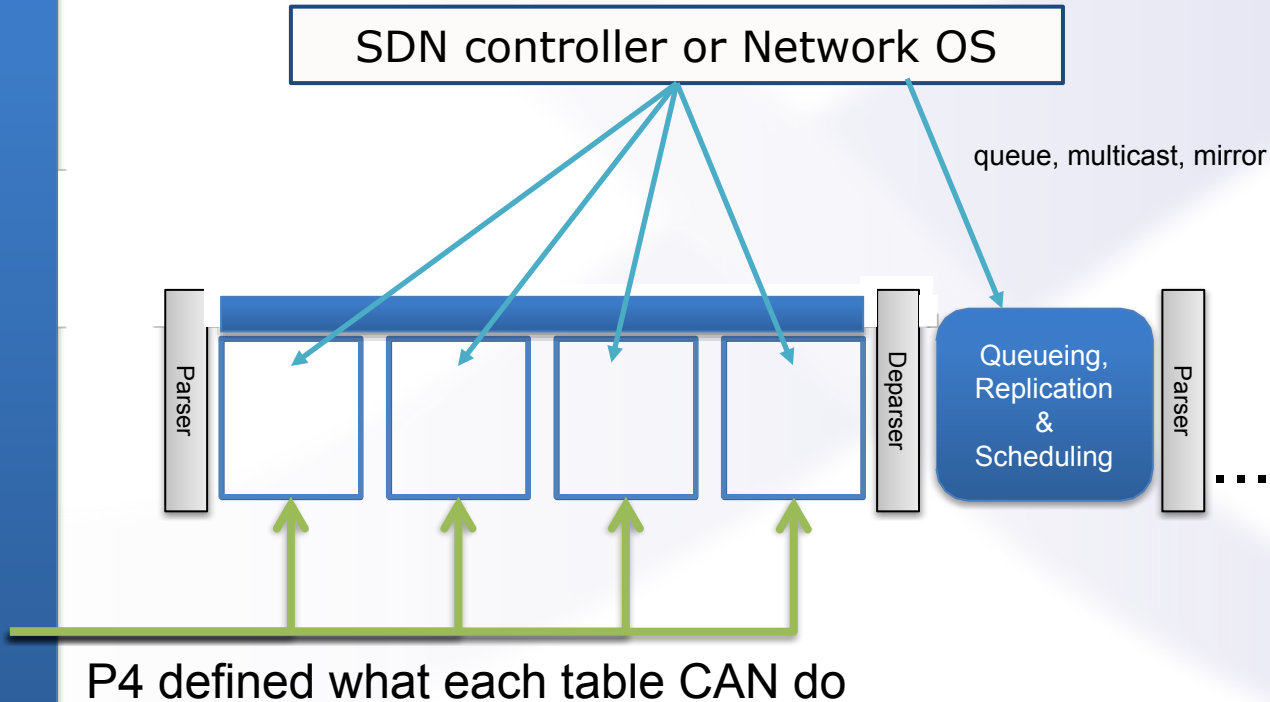
program.p4

Data Declarations

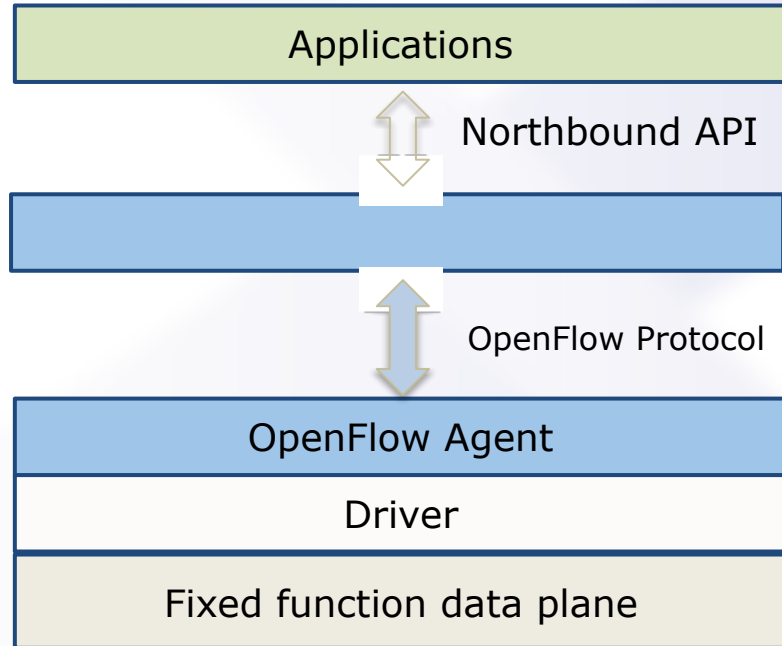
Pars

Table + Control Flow Program

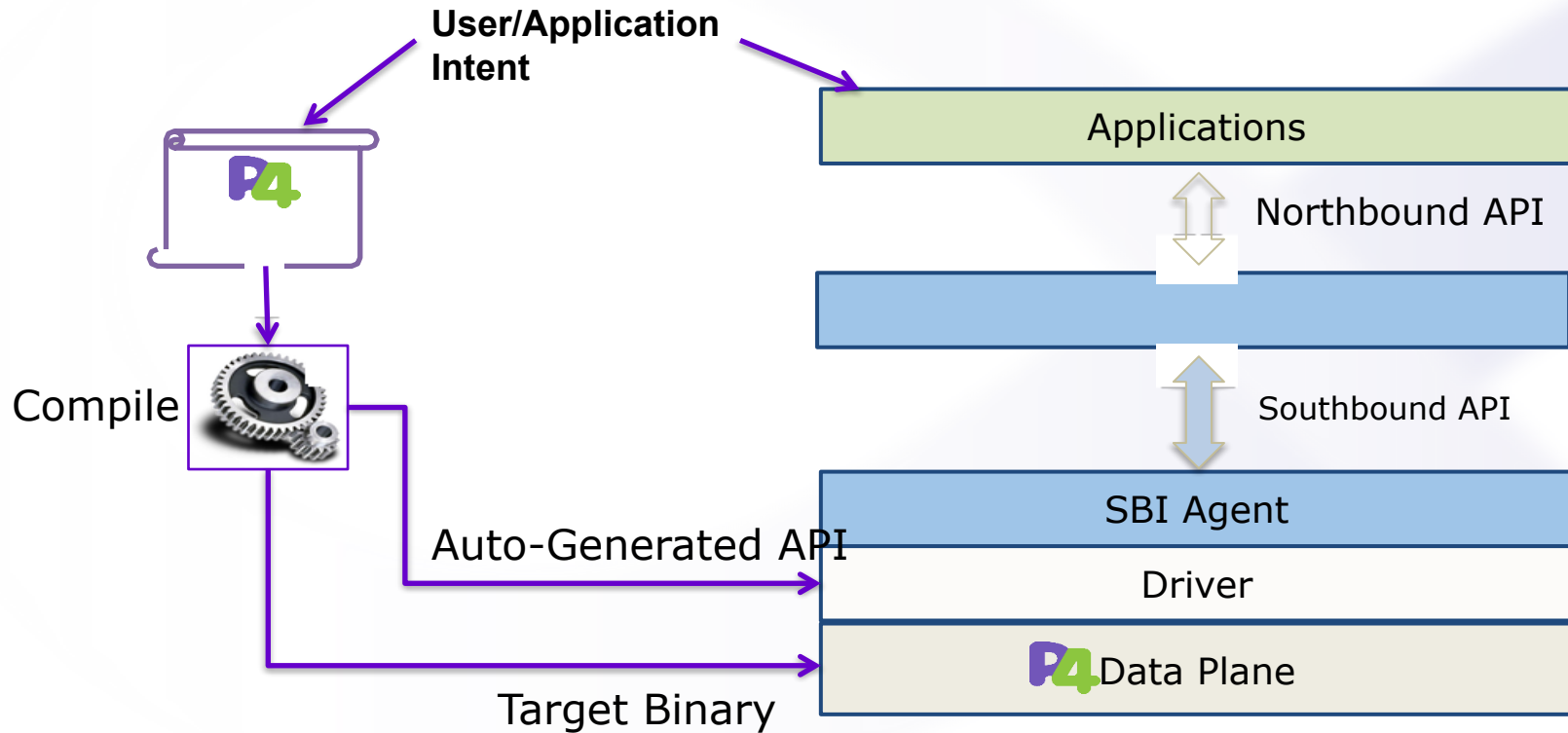
Control plane or NOS decides **switch runtime behavior**



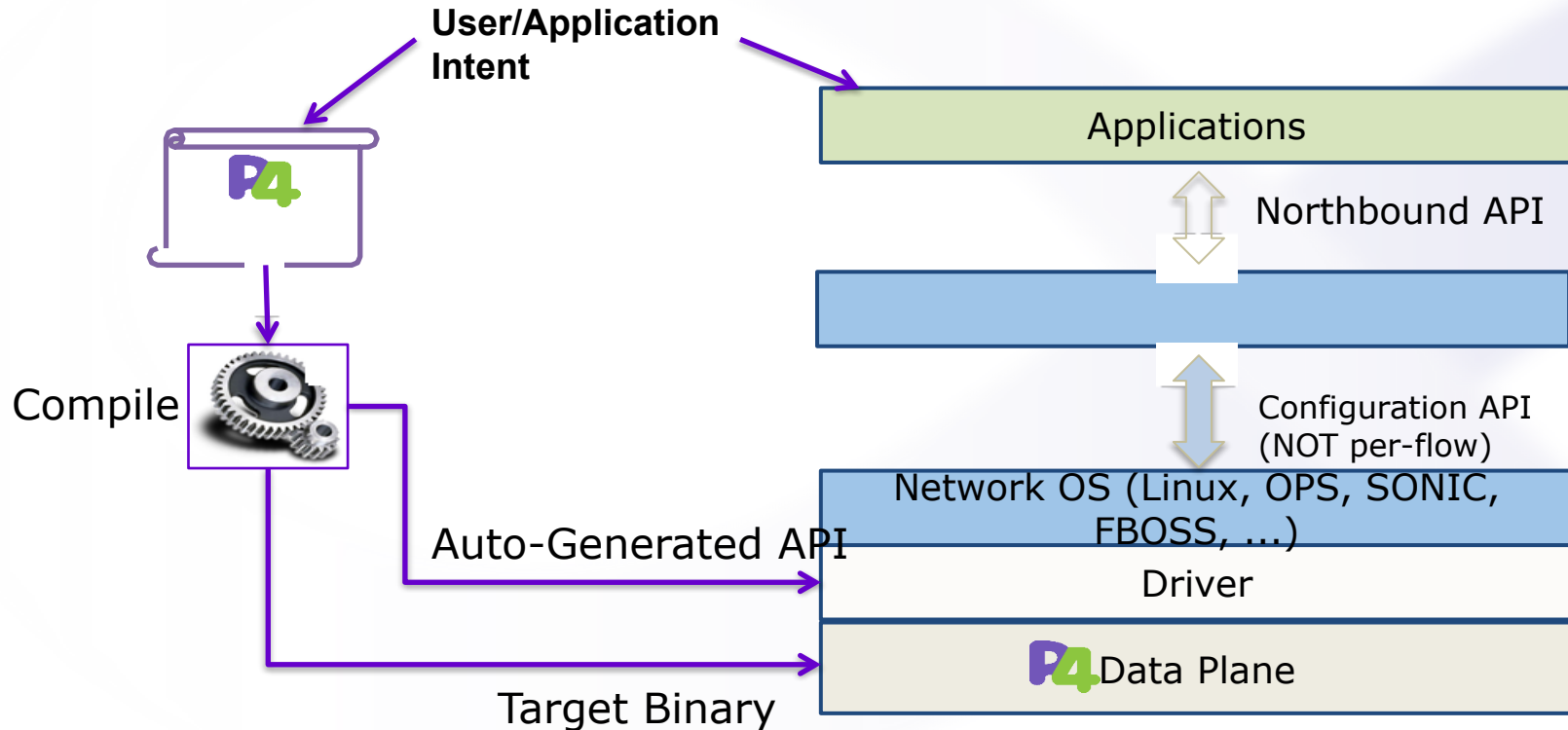
P4 & OpenFlow: Traditional SDN before P4



P4 with OpenFlow



P4 with Network OS



P4 Constructs

- P4 Spe

P4 Language Components

- **Data declarations**
 - Packet Headers and Metadata
- **Parser Programming**
 - Parser Functions (Parser st
 - Checksum Units
- **P**
 - - Primitive and
 - Counters, Meters, Registers
 - Tables
 - Match keys
 - Attributes
 - Control Functions (Imperative Programs)

No: pointers, loops, recursion, floating point

Headers and Fields (Packet)

Example: Declaring packet headers

```
header_type ethernet_t {  
    fields {  
        dstAddr    : 48;  
        srcAddr    : 48;  
        etherType  : 16;  
    }  
}
```

Header Type
Declar

```
header_type vlan_tag_  
    fields {  
        pcpi       : 3;  
        cfi        : 1;  
        vid        : 12;  
        etherType  : 16;  
    }  
}
```

Actual Header
Instantiation

```
header ethernet_t ethernet;  
header vlan_tag_t vlan_tag[3];
```

Handy Arrays for
Header Stacks

Headers and Fields (Metadata)

Example: Declaring Metadata

```
header_type ingress_metadata_t {  
    fields {  
        /* Inputs */  
        ingress_port      : 9; /*  
        packet_length     : 16; /*  
  
        /* Outputs from Ingress Pipeline */  
        egress_spec       : 16;  
        queue_id          : 9;  
    }  
}  
  
metadata ingress_metadata_t ingress_metadata;
```

Metadata is a header
too

Actual Metadata
Instantiation

Metadata vs. Packet Headers

- **Layout definition**

- Packet header declarations define both the fields and the actual layout in the packet.
- Layout is not defined for m

- **B**

-
- No special r

- **Validity**

- Packet headers are valid only if present in the packet
- Metadata is ALWAYS valid
 - Default value is either 0 or can be specified explicitly

- **Acceptable fields**

- Packet headers can contain calculated and variable length fields

Variable-Length Fields

Example: Declaring IPv4 packet header

```
header_type ipv4_t {  
    fields {  
        version      : 4;  
        ihl           : 4;  
        diffserv      : 8;  
  
        flags  
        fragOffset  
        ttl           : 8;  
        protocol      : 8;  
        hdrChecksum    : 16;  
        srcAddr        : 32;  
        dstAddr        : 32;  
        options        : *;  
    }  
    length      : (ihl << 2);  
    max_length  : 60;  
}
```

Variable-length Field

Calculated, based on
another field

Defining a Parser Tree

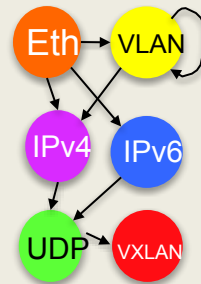
Example: Simple Parser for L2/L3 Packets

```
header ethernet_t ethernet;  
header vlan_tag_t vlan_tag[2];  
header ipv4_t ipv4;  
header ipv6_t ipv6;
```

```
parser  
  ext  
  ret  
  
    0x0800  
    0x86DD  
  default      : ingress;  
}
```

```
parser parse_vlan_tag {  
  extract(vlan_tag[next]);  
  return select(latest.etherType) {  
    0x8100 mask 0xEFFF : parse_vlan_tag;  
    0x0800             : parse_ipv4;  
    0x86DD             : parse_ipv6;  
    default           : ingress;  
  }  
}
```

This is not a reserved word, but a name of the Control Flow Function



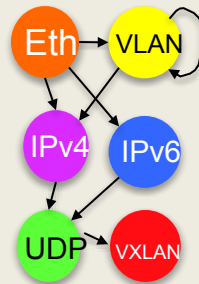
Defining a Parser Tree (cont.)

Example: Simple Parser for L2/L3 Packets

```
header ethernet_t ethernet;  
header vlan_tag_t vlan_tag[2];  
header ipv4_t ipv4;  
header ipv6_t ipv6;
```

```
parser  
  ext  
  ret  
  
    0x0800  
    0x86DD  
  default      : ingress;  
}  
  
parser parse_vlan_tag {  
  extract(vlan_tag[next]);  
  return select(latest.etherType) {  
    0x8100 mask 0xEFFF : parse_vlan_tag;  
    0x0800             : parse_ipv4;  
    0x86DD             : parse_ipv6;  
    default            : ingress;  
  }  
}
```

```
parser parse_ipv4 {  
  extract(ipv4);  
  return ingress;  
}
```



Multi-field select statement

Example: Ipv4 Header Parsing

```
parser parse_ipv4 {  
    extract(ipv4);  
    set_metadata(ipv4_metadata.lkp_ipv4_sa, ipv4.srcAddr);  
    set_metadata(ipv4_metadata.lkp_ipv4_  
    set_metadata(l3_metadata.lkp_ip_prot  
se  
  
re  
  
    0x0000506 : p  
    0x0000511 : p  
    default   : ingress;  
}
```

Metadata can be
initialized by the
parser

Fields are joined
for a match

Deparsing (Serializing packet headers)

- **Fundamental assumption of P4**

- The device must be able to parse any packet it can produce

- **Consequence**

- Packet headers can be read after ingress parser definition

○

- Example: insert my_header after udp

- ```
parser parse_udp {
 extract(udp);
 return select(latest.dst_port) {
 0x0 mask 0x00 : ingress;
 default : parse_my_header;
 }
}
```

Ingress parser will  
always transit to  
ingress

Parser tree has a  
branch to my\_header  
for deparsing

# P4 Language Components

---

- Data declarations
- Parser Programming

- **P**



- Primitive and
    - Counters, M

- Tables

- Match keys
    - Attributes

- Control Functions

# Actions

- **Primitive actions**

- no\_op, drop
- modify\_field, modify\_field\_with\_hash\_based\_offset
- add, add\_to\_field
- add\_header, remove\_header
- 
- 
- generate\_digest
- truncate
- resubmit, recirculate, clone{ \_i2i, \_e2i, \_i2e, \_e2e }

- **Compound actions**

```
action route_ipv4(dst_port, dst_mac, src_mac, vid) {
 modify_field(standard_metadata.egress_spec, dst_port);
 modify_field(ethernet.dst_addr, dst_mac);
 modify_field(ethernet.src_addr, src_mac);
 modify_field(vlan_tag.vid, vid);
 add_to_field(ipv4.ttl, -1);
}
```

# Arithmetic and Logical Primitives

- **The current standard (v1.0.2)**

- Primitive actions

- Standard: `add()`, `add_to_field()`
    - Additional: `subtract()`, `subtract`  
`shift_right()`, ...
    -

`r()`, `bit_xor()`, `shift_left()`,

- **Developing**

- Expressions with `+`, `-`, `&`, `|`, `^`, `~`, `<<`, `>>`, etc.

- `modify_field(ipv4.ttl, ipv4.ttl - 1)`

- Specific targets might restrict expression complexity

# Action Execution Semantics

- All actions within a compound action are assumed to be executed sequentially

```
action parallel_test() {
 modify_field(hdr.fieldA, 1);
 modify_field(hdr.fieldB, hdr.f
```



- This is an important specification change
  - Up to version 1.0.2 action execution was parallel
  - After 1.0.2 action execution is sequential
- The maximum number of steps supported for a compound action is target-dependent

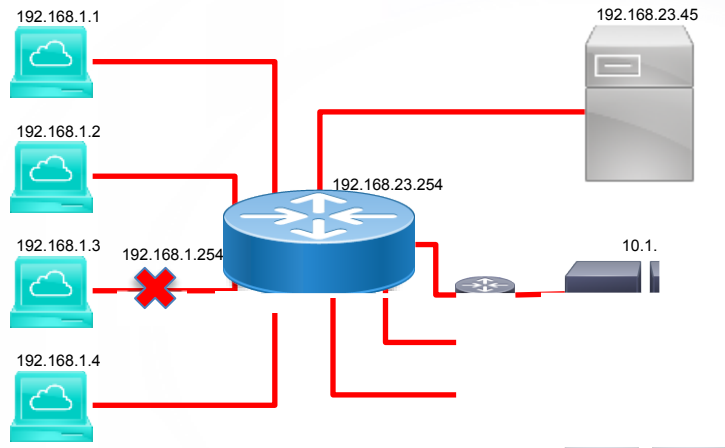


# Match-Action Tables

- The most fundamental units of the Match-Action Pipeline
- P4 defines
  - What to match on and match type
  - A list of *possible* actions
  - -
- In runtime,
- An entry contains:
  - A specific key to match on
  - A **single** action
    - to be executed when a packet matches the entry
  - (Optional) action data

ws)

# Example: IPv4 Processing



| Key            | Action         | Action Data                    |
|----------------|----------------|--------------------------------|
| 192.168.1.1    | I3_switch      | port= mac_da= mac_sa= vlan=    |
| 192.168.1.2    | I3_switch      | port= mac_da= mac_sa= vlan=... |
| 192.168.1.3    | I3_drop        |                                |
| 192.168.1.254  | I3_I2_switch   | port=                          |
|                |                |                                |
|                |                |                                |
| 192.168.1.0/24 | I3_I2_switch   | port=                          |
| 10.1.2.0/22    | I3_switch_ecmp | ecmp_group=                    |

## • P4 Program

- Defines the format of the table
  - Key Fields
  - Actions
  - Action Data

entries with

specific information

- Based on the configuration
- Based on automatic discovery
- Based on protocol calculations

## • Data Plane (populated table)

- Performs the lookup
- Executes the chosen action

# Defining Actions

```
action l3_switch(port, mac_da, mac_sa, vlan) {
 modify_field(metadata.egress_spec, port);
 modify_field(ethernet.dstAddr, mac_da);
 modify_field(ethernet.srcAddr, mac_sa);
 modify_field(vlan_tag[0].vlanid, vlan);
 modify_field(ipv4.ttl, ipv4.ttl - 1);
}

action l3_l2_switch(port) {
 mod
}

action l3_drop() {
 drop();
}

action l3_switch_nexthop(nexthop_index) {
 modify_field(l3_metadata.nexthop, nexthop_index);
 modify_field(l3_metadata.nexthop_type, NEXTHOP_TYPE_SIMPLE);
}

action l3_switch_ecmp(ecmp_group) {
 modify_field(l3_metadata.nexthop, ecmp_group);
 modify_field(l3_metadata.nexthop_type, NEXTHOP_TYPE_ECMP);
}
```

# Match-Action Table (Exact Match)

**Example:** A typical L3 (IPv4) Host table

```
table ipv4_host {
 reads {
 ingress_metadata.vrf : exact;
 ipv4.dstAddr : exact;
 }
 ac

 l3_switch_nex
 l3_switch_ecm
 l3_drop;
}
size : HOST_TABLE_SIZE;
}
```



| vrf | ipv4.dstAddr | action         | data                     |
|-----|--------------|----------------|--------------------------|
| 1   | 192.168.1.10 | l3_switch      | port_id= mac_da= mac_sa= |
| 100 | 192.168.1.10 | l3_l2_switch   | port_id=<CPU>            |
| 1   | 192.168.1.3  | l3_drop        |                          |
| 5   | 10.10.1.1    | l3_switch_ecmp | ecmp_group=127           |

# Match-Action Table (Longest Prefix Match)

**Example:** A typical L3 (IPv4) Routing table

```
table ipv4_lpm {
 reads {
 ingress_metadata.vrf : exact;
 ipv4.dstAddr : lpm;
 }
 ac

 13_ecmp;
 13_drop;
}
size : 65536;
}
```

Different fields can  
use different  
match types

Prefix also serves  
as a priority  
indicator

| vrf | ipv4.dstAddr / prefix | action            | data              |
|-----|-----------------------|-------------------|-------------------|
| 1   | 192.168.1.0 / 24      | l3_l2_switch      | port_id=64        |
| 10  | 10.0.16.0 / 22        | l3_ecmp           | ecmp_index=12     |
| 1   | 192.168.0.0 / 16      | l3_switch_nexthop | nexthop_index=451 |
| 1   | 0.0.0.0 / 0           | l3_switch_nexthop | nexthop_index=1   |

# Match-Action Table (Ternary Match)

**Example:** A typical L3 (IPv4) Routing table

```
table ipv4_lpm {
 reads {
 ingress_metadata.vrf : ternary;
 ipv4.dstAddr : ternar
 }
 ac

 13_ecmp;
 13_drop;
}
size : 65536;
}
```

Ternary tables require  
an explicit  
specification of entry  
priority

| Prio | vrf / mask   | ipv4.dstAddr / mask           | action            | data             |
|------|--------------|-------------------------------|-------------------|------------------|
| 100  | 0x001/0xFFFF | 192.168.1.5 / 255.255.255.255 | l3_switch_nexthop | nexthop_index=10 |
| 10   | 0x000/0x000  | 192.168.2.0/255.255.255.0     | l3_switch_ecmp    | ecmp_index=25    |
| 10   | 0x000/0x000  | 192.168.3.0/255.255.255.0     | l3_switch_nexthop | nexthop_index=31 |
| 5    | 0x000/0x000  | 0.0.0.0/0.0.0.0               | l3_l2_switch      | port_id=64       |

# Match Types

---

- **Exact**

- port\_index : exact

- **Ternary**

- ethernet.srcAddr : ternary

- **L**

- i

- **Range**

- udp.dstPort : range

- **Valid**

- vlan\_tag[0] : valid

# Table Miss

---

- **Each table can have a Default Action**

- Chosen by the Control Path at runtime from the list of table Actions
  - P4 Program does not have an indication which action (and which action data) will be the default

- **W**

- control path
- If no Default Action has been specified, it is **no\_op()**



# Stateful Objects

---

- Count

# What are stateful objects

---

- **Stateful objects keep their state between packets**
  - Metadata and packet headers are **stateless**
    - They are re-initialized for each packet
  - Counters, Meters and Regi
    - 
    - 
    -

# Direct Counters

## A counter per table entry

```
counter ip_acl_stats {
 type : packets_and_bytes;
 direct : ip_acl;
}
```

```
table ip_acl {
 rea

 13_metadata.1k
 13_metadata.1k
}
actions {
 nop;
 acl_log;
 acl_deny;
 acl_permit;
 acl_mirror;
 acl_redirect_nexthop;
 acl_redirect_ecmp;
}
size : INGRESS_IP_ACL_TABLE_SIZE;
}
```

| table ip_acl   |            |             | counter ip_acl_stats |  |
|----------------|------------|-------------|----------------------|--|
| Match Fields   | Action Sel | Action Data | Counter              |  |
| ABCD_xxxx_0123 | acl_deny   |             | counter A            |  |
| matched entry  | acl_permit | 8b 8b       | pkt/byte counts      |  |
|                |            |             |                      |  |
|                | p          |             | counter Z            |  |

# Indirect Counters

## Flexibly linked counters

```
counter ingress_bd_stats {
 type : packets_and_bytes;
 instance_count : BD_STATS_TABLE_SIZE;
}
```

Different VLANs (BDs) can share the same counter

```

action
 mo
 co
}

```

```
table port_vlan {
 reads {
 ingress_metadata.ingress_port : exact;
 vlan_tag[0] : valid;
 vlan_tag[0].vlan_id : exact;
 }
 actions {
 set_bd;
 }
}
```

| table port_vlan      |               |             |                        |
|----------------------|---------------|-------------|------------------------|
| Match Fields         | Action Sel    | Action Data |                        |
| ABCD_0123            | set_bd        | bd          | bd_stat_index          |
|                      | set_bd        | bd          | bd_stat_index          |
| <b>matched entry</b> | <b>set_bd</b> | <b>bd</b>   | <b>bd_stat_index A</b> |
|                      | set_bd        | bd          | bd_stat_index          |
|                      | set_bd        | bd          | bd_stat_index          |
|                      | set_bd        | bd          | bd_stat_index A        |
| BA8E_F007            | set_bd        | bd          | bd_stat_index          |

```
counter
ingress bd stats
```

pkt/byte counts

# Meters

- **Declaration is similar to counters**

- Action: `execute_meter(meter_array, meter_index, color_destination)`

```
/* Direct Meter */
```

```
meter acl_meter {
```

```
 t
```

```
 d
```

```
 r
```

```
}
```

```
/* Indirect Meter Array */
```

```
meter bd_meter {
```

```
 type: bytes;
```

```
 instance_count: 1000;
```

```
}
```

```
action do_bd_meter(meter_index) {
```

```
 execute_meter(bd_meter, meter_index, metadata.color);
```

```
}
```

Color Coding:

0 – Green

1 – Yellow

2 -- Red

# Registers

- Declaration is similar to indirect counters

- Actions

- register\_read(register\_array, register\_index, destination\_field)
    - register\_write(register\_array, register\_index, value)

```
/* Register Array (Indirect) */
regist
 wi
 sta
 instance_count: 1
}

action get_flow_age(flow_index) {
 register_read(last_syn, flow_index, metadata.flow_start_time);
 modify_field(metadata.flow_age,
 metadata.flow_start_time - metadata.ingress_global_stamp);
}

action start_new_flow(flow_index) {
 register_write(last_syn, flow_index, metadata.ingress_global_timestamp);
}
```

# Control Flow Functions

---

- **Primitives**

- Perform a table lookup: **apply**
- **if/else** statement
- **apply** with the case clause

- **S**

- 

- **Standard c**

- ingress() – Ingress Pipeline processing
- egress() – Egress Pipeline processing

- **User-defined control functions**

# Standard Control Functions

- **ingress()** control function starts processing
  - remember “return ingress;” statement in the parser functions
- **egress()** control function is called implicitly from the Packet Replication Engine

```
contr
 a
 a
 apply(dmac);
}

control egress {
 apply(vlan_tag_removal);
}
```



# User-Defined Control Functions

- **Help improve code readability**
  - No specific performance advantages: the code is flattened by the compiler
- **No parameters are accepted**

```
control assign_vlan {
 ap
 ap
 ap
 apply(port_vlan)
 apply(resolve_vl
}

control ingress {
 . . .
 if (!valid(vlan_tag[0])) {
 assign_vlan();
 }
 . . .
}
```

# If/Else Branching

## Example: Separate Ipv4 and IPv6 Processing Paths

```
if ((l3_metadata.lkp_ip_type == IPTYPE_IPV4) and (ipv4_metadata.ipv4_unicast_enabled == TRUE)) {
 process_ipv4_racl();
 process_nat();
 process_ipv4_urpf();
 process_ipv4_fib();
} else
 if
 process_ipv6
 process_ipv6
 process_ipv6_fib();
 }
```

# Action Branching

**Example:** Use per-router-mac decapsulation

```
table router_mac {
 reads {
 l2_metadata.lkup_dst_mac : ternary;
 l2_metadata.bd : ternary;
 ingress_metadata.src_port: ternar
 }
 act

 enable_ipv6_lo
 enable_mpls_de
 enable_mim_decap;
}

control process_router_mac_lookup {
 apply(router_mac) {
 enable_ipv4_lookup { process_ipv4_fib(); }
 enable_ipv6_lookup { process_ipv6_fib(); }
 enable_mpls_decap { process_mpls_label_lookup(); }
 /* etc. */
 }
}
```

# Miss branching

```
action on_miss() {}
```

```
table ipv4_fib {
 reads {
 . . .
```

```
 l3_switch_ecmp;
 on_miss;
 }
}
```

We choose to use only  
this action as the default

```
control process_ipv4_fib {
 apply(ipv4_fib) {
 on_miss {
 apply(ipv4_fib_lpm);
 }
 }
}
```

# Executing actions in the control flow

```
control process_counters {
 if (my_meta.drop_packet == 0) {
 count(bd_counter,
 metadata.bd_counter_index);
 count(vrf_counter,
 metadata.vrf_counter_index);
 }
}
```

```
action update_counters() {
 count(bd_counter,
 metadata.bd_counter_index);
 count(vrf_counter,
 metadata.vrf_counter_index);
}
```

```
counters {
```

```
 if (my_meta.drop_packet == 0) {
 apply(do_process_counters);
 }
}
```

- **Actions cannot be directly referenced in the control flow functions**
  - Instead, they need to be “wrapped” into tables
- **Tables without keys can be used to implement unconditional execution**
  - They always miss and hence the desired action needs to be set as a default

# P4 Compiler Ov

# Modular Compiler Overview

- **Single Front-End (p4-hlir)**

- Translates P4 code into High-Level Intermediate Representation (HLIR)

- Similar to AST (Abstract Syntax Trees)
- Currently represented as a hie
- Frees backend developers fro

is and target-independent

- 

- **Multiple ba**

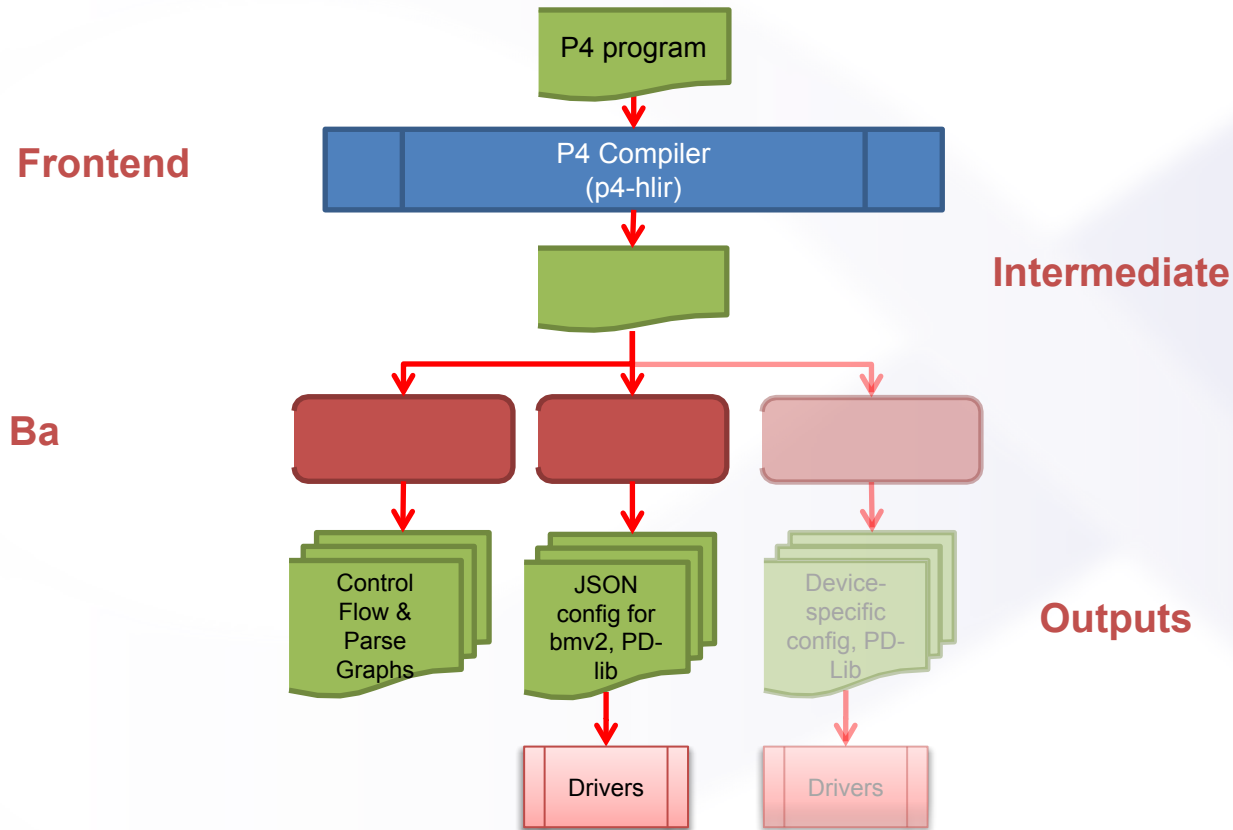
- Code generators for various targets

- Software Switch Model (p4c-bm)
- Network Interface Cards
- Packet Processors / NPUs
- FPGAs, GPUs, ASICs

- Validators and graph generators

- Run-time API generators

# P4 Modular Compiler



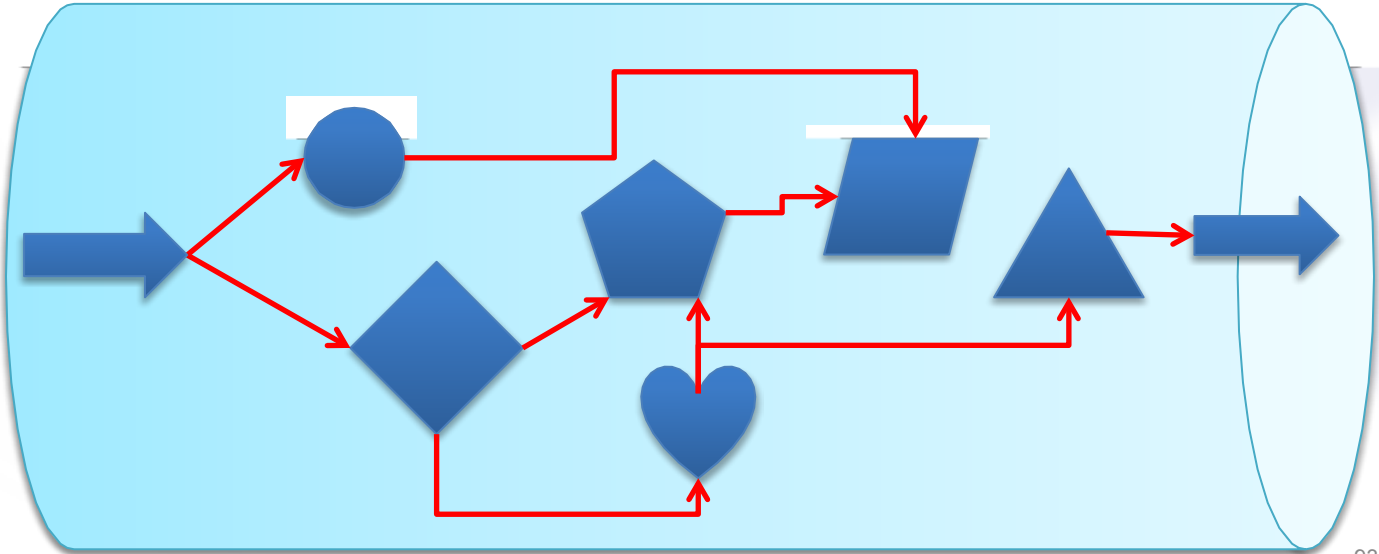


# **Automatic API Generation**

---

# Network Device API Basics

- **Object Definitions (Schema)**
  - Reflects the object properties and methods
- **Object Relationships (Behavior)**
  - The quality of th ent on how well



# P4 is an Ideal Base for a Network APIs

---

- **Clearly defined objects**

- Tables
- Counters
- Meters
- 

- **U**

- Control Flo

- **Idea:**

- Each of fundamental P4 objects has a “natural” schema

# Tables

- **Uniform representation**

- Primary key: Entry ID
- Match Fields
- Action
- Action Data
  - Depends on the action

- **Operations**

- En
  - (
- Ent
  - (Entry ID, Action, Action Data) →
- Entry Lookup
  - (Match Fields, [Action, Action Data]) → Entry ID
- Table Traverse
  - → [ EntryID0, EntryID1, ... EntryIDn ]
- Table Default\_Action Set
  - (Action, Action Data) →
- Table Default\_Action Get
  - → (Action, Action Data)
- Table Default Action Clear
  - →

| EntryID | Match Fields | Action Sel | Action Data |
|---------|--------------|------------|-------------|
|         | ABCD_0123    | action A   |             |
|         |              |            |             |
|         |              |            |             |
|         |              |            |             |
|         |              |            |             |
|         |              |            |             |
|         |              |            |             |
|         |              |            |             |

- Table Occupancy Get
- Table Clear

# Example API. Match & Action Specs

## myprog.p4

```
action a1(p11, p12) {...}
action a2(p21, p22, p23) {...}
action a3() {...}

table t1 {

 h

}
```

## pd\_myprog.h

```
typedef struct p4_pd_myprog_a1_action_spec {
 <type> p11;
 <type> p12;

} p4_pd_myprog_a1_action_spec_t;

typedef struct p4_pd_myprog_a3_action_spec {
} p4_pd_myprog_a3_action_spec_t;

typedef struct p4_pd_myprog_t1_match_spec {
 <type> meta_f1;
 <type> meta_f2;
 <type> meta_f2_mask;
 uint8_t h1_valid;
} p4_pd_myprog_t1_match_spec_t;
```

exact: f  
ternary: f and f\_mask  
lpm: f and f\_prefix\_len  
valid: f\_valid  
range: f\_min and f\_max

# Example API. Entry Add

## myprog.p4

```
action a1(p11, p12) {...}
action a2(p21, p22, p23) {...}
action a3() {...}

table t1 {

 h
}
actions {
 a1;
 a2;
 a3;
}
}
```

## pd\_myprog.h

```
p4_pd_status_t p4_pd_myprog_t1_entry_add_with_a1(
 p4_pd_target_t device_target,
 p4_pd_session_t session_handle,
 priority,
 spec_t *match_spec,

 t,
 le,

 p4_pd_priority_t priority,
 const p4_pd_myprog_t1_match_spec_t *match_spec,
 const p4_pd_myprog_a2_action_spec_t *action_spec,
 p4_pd_entry_handle_t *entry_hdl);

p4_pd_status_t p4_pd_myprog_t1_entry_add_with_a3(
 p4_pd_target_t device_target,
 p4_pd_session_t session_handle,
 p4_pd_priority_t priority,
 const p4_pd_myprog_t1_match_spec_t *match_spec,
 const p4_pd_myprog_a3_action_spec_t *action_spec,
 p4_pd_entry_handle_t *entry_hdl);
```

# Example API. Entry Modify

## myprog.p4

```
action a1(p11, p12) {...}
action a2(p21, p22, p23) {...}
action a3() {...}

table t1 {

 h
}
actions {
 a1;
 a2;
 a3;
}
}
```

## pd\_myprog.h

```
p4_pd_status_t p4_pd_myprog_t1_entry_modify_with_a1(
 p4_pd_target_t device_target,
 p4_pd_session_t session_handle,
 entry_hdl,
 _spec_t *action_spec);

 le,

 const p4_pd_myprog_a2_action_spec_t *action_spec);

p4_pd_status_t p4_pd_myprog_t1_entry_modify_with_a3(
 p4_pd_target_t device_target,
 p4_pd_session_t session_handle,
 p4_pd_entry_handle_t entry_hdl,
 const p4_pd_myprog_a3_action_spec_t *action_spec);
```

# Example API. Entry Delete and Lookup

## myprog.p4

```
action a1(p11, p12) {...}
action a2(p21, p22, p23) {...}
action a3() {...}
```

```
 m
 h
 }
 actions {
 a1;
 a2;
 a3;
 }
}
```

## pd\_myprog.h

```
p4_pd_status_t p4_pd_myprog_t1_entry_delete(
 p4_pd_target_t device_target,
 p4_pd_session_t session_handle,
 entry_hdl);
```



# Example API. Entry Get

## myprog.p4

```
action a1(p11, p12) {...}
action a2(p21, p22, p23) {...}
action a3() {...}

table t1 {

 h
}
actions {
 a1;
 a2;
 a3;
}
}
```

## pd\_myprog.h

```
typedef enum {
 P4_PD_MYPROG_ACTION_A1,
 P4_PD_MYPROG_ACTION_A2,
 . . .
} p4_pd_myprog_action_spec_t;

p4_pd_status_t p4_pd_myprog_t1_entry_get(
 p4_pd_target_t device_target,
 p4_pd_session_t session_handle,
 p4_pd_entry_handle_t entry_hdl,
 p4_pd_myprog_t1_match_spec_t *match_spec,
 p4_pd_myprog_actions_t *action,
 p4_pd_myprog_action_spec_t *action_spec_t);
```

# Example API. Default Action APIs

## myprog.p4

```
action a1(p11, p12) {...}
action a2(p21, p22, p23) {...}
action a3() {...}

table t1 {

 h
}

actions {
 a1;
 a2;
 a3;
}

}
```

## pd\_myprog.h

```
p4_pd_status_t p4_pd_myprog_t1_set_default_action_a1(
 p4_pd_target_t device_target,
 p4_pd_session_t session_handle,
 _spec_t *action_spec);

;

p4_pd_status_t p4_pd_myprog_t1_set_default_action_a3(
 p4_pd_target_t device_target,
 p4_pd_session_t session_handle,
 const p4_pd_myprog_a3_action_spec_t *action_spec);

p4_pd_status_t p4_pd_myprog_t1_clear_default_action(
 p4_pd_target_t device_target,
 p4_pd_session_t session_handle);
```

# Program-Dependent API style

Uniform structure and small number of P4 objects allow APIs to be generated automatically

## myprog.p4

```
action a1(p11, p12) {...}
action a2(p21, p22, p23) {...}
action a3() {...}
```

```
actions {
 a1;
 a2;
 a3;
}
```

## pd\_myprog\_style1.h

```
typedef enum {
 P4_PD_MYPROG_ACTION__A1,
 P4_PD_MYPROG_ACTION__A2,
 . . .
 P4_PD_MYPROG_ACTION__MAX
} p4_pd_myprog_actionid_t;
```

```
LE__MAX
_id_t;
```

```
typedef enum {
 P4_PD_MYPROG_FIELD__META_F1,
 P4_PD_MYPROG_FIELD__META_F2,
 P4_PD_MYPROG_FIELD__META_F2__MASK,
 P4_PD_MYPROG_FIELD__H1__VALID,
 P4_PD_MYPROG_FIELD__P11,
 P4_PD_MYPROG_FIELD__P12,
 P4_PD_MYPROG_FIELD__P22,
 P4_PD_MYPROG_FIELD__P23,
 . . .
 P4_MD_MYPROG_FIELD__MAX
} p4_pd_myprog_field_id_t;
```

# github.com/p4lang

---

- switch

am

- Project summaries: [link](#)

# BMv2 Primitives

---

- standard primitives [https://github.com/p4lang/p4-hlir/blob/master/p4\\_hlir/frontend/primitives.json](https://github.com/p4lang/p4-hlir/blob/master/p4_hlir/frontend/primitives.json)
- bmv2 specific primitives [lang/p4c-b](#)

# Intrinsic Metadata, provided by BMv2 switch

- If one defines all these fields, all the simple\_switch features will be supported, so it is recommended to define these fields in every program (to avoid a headache).

```
header_type intrinsic_metadata_t {
 fields {
 ingress_global_timestamp : 48; // ingress timestamp, in microseconds
 mcast_grp : 4; // to be set in the ingre
 egress_rid : 4; // replication id, availab
 mcast_hash : 16; // unused
```

```
 }
}
```

```
metadata intrinsic_metadata_t intrinsic_metadata;
```

```
header_type queueing_metadata_t {
 fields {
 enq_timestamp : 48; // in microseconds
 enq_qdepth : 16;
 deq_timedelta : 32;
 deq_qdepth : 16;
 }
}
```

```
metadata queueing_metadata_t queueing_metadata;
```

**Thank you**

